FOODMITRA

Multi-Party Payment Architecture & Admin Flow

Customer • Restaurant/Vendor • Rider • Tiffin House • Admin

Step-by-step implementation blueprint for a multi-vendor food delivery platform

Core decision

FoodMitra ko sirf ek “payment status” field par mat chalana. Order, payment, vendor allocation, rider earning, refund, settlement aur reconciliation ko alag records ke roop mein track karo. Actual funds movement ke liye approved marketplace/split-settlement payment provider integrate karo.

Is document ka scope

Ye architecture aur implementation plan hai—legal, accounting, tax ya payment-provider approval ka substitute nahi. Provider se confirm karo ki FoodMitra ka business model, vendor onboarding, settlement hold/cycle, rider payouts aur tiffin-house use case unke product mein approved/supported hain.

1. Sabse pehle payment model decide karo

- Customer app: customer ek payment karega; backend payment ko parent checkout aur vendor-wise sub-orders mein map karega.
- Restaurant/vendor: har vendor ka apna vendorId aur provider-side linked/vendor accountId hoga. Allocation isi stable ID se attach hogi—restaurant ke display name se nahi.
- Tiffin House app: tiffin house ko bhi ek vendor type ke roop mein model karo. One-time meals aur weekly/monthly subscription/prepaid plans ke payment rules alag rakho.
- Rider app: rider earning delivery complete hone, distance/zone/rules, incentives aur adjustments ke basis par calculate hogi. Rider payout ko vendor payment se alag track karo, jab tak chosen provider/contract explicitly rider ko same split flow mein support na kare.
- Admin: onboarding approvals, commission rules, settlement review, refunds, disputes, reconciliation aur audit logs manage karega. Admin ko paid status manually edit karke “payment successful” banane ki permission nahi honi chahiye.

Recommended default

Use ek marketplace / split-settlement capable payment provider, jaise Razorpay Route ya Cashfree Easy Split, subject to eligibility, commercial terms, KYC and provider approval. Dono ke official docs vendor-linked accounts, payment split/transfer aur settlement/refund/reconciliation capabilities describe karte hain [1][2].

2. High-level architecture (boxes mein)

**CUSTOMER APP	Restaurant/Vendor App	Rider App	Tiffin House App**
↓ HTTPS API
FOODMITRA BACKEND (Node.js / Express)
Auth + Order Service + Pricing/Commission Engine + Payment Service + Refund Service
Settlement/Ledger Service + Webhook Handler + Admin RBAC + Audit Log
↓
DATABASE: Orders • SubOrders • Payments • Allocations • Ledger • Settlements • Refunds • Webhook Events
↓ Server-to-server APIs / signed webhooks
PAYMENT PROVIDER: Customer collection + approved marketplace split/transfers + settlement status
↓
BANK ACCOUNTS: FoodMitra/platform share • Restaurant/vendor • Tiffin house • Rider payout (provider-supported flow)

Important

React Native app ko kabhi payment gateway secret key, service-account private key ya privileged payout credentials mat do. Payment orders, split instructions, refund requests aur admin actions backend se hi honge.

3. Real order ka end-to-end flow

Step 1 — Checkout validation: Backend cart ke prices, available items, vendor ownership, delivery fee, coupon, tax configuration aur final payable amount dobara calculate kare. App se aaya total blindly trust na kare.

Step 2 — Parent order + sub-orders: Parent checkout/order banao. Cart mein 2 restaurants hain to 2 separate sub-orders banao: SubOrder A -> Restaurant A; SubOrder B -> Restaurant B. Tiffin-house vendor ke liye bhi same pattern.

Step 3 — Price snapshot + allocation: Checkout ke waqt item prices, commission percentage/fixed amount, discount funding (platform ya vendor), fees aur net payable ka immutable snapshot save karo. Baad mein commission config change ho to purane order ka hisaab na badle.

Step 4 — Provider payment order: Backend provider API se payment order/session create kare using a unique idempotency key. Client ko sirf required checkout/public data bhejo.

Step 5 — Customer payment: Customer UPI/card/netbanking ya supported method se provider checkout complete kare.

Step 6 — Verified webhook: Provider webhook aane par signature verify karo, event ID deduplicate karo, amount/currency/orderId match karo. Client ke success callback ko final proof mat mano.

Step 7 — Payment captured: Verified capture ke baad Payment record update karo; parent order aur relevant sub-orders ko paid state mein lao. Payment capture na hui ho to vendor ko paid order fulfill karne ke liye mat bhejo.

Step 8 — Fulfilment: Restaurant/tiffin-house order accept aur prepare kare; rider assign/pickup/delivery updates alag order-status workflow mein rahen.

Step 9 — Settlement eligibility: Policy ke hisaab se delivery complete, cancellation/refund checks aur any configured hold/cooling period ke baad vendor payable settlement ke liye eligible ho. Provider hold/defer capability aur rules pehle confirm karo.

Step 10 — Transfer and reconcile: Provider API transfer/split initiate ya scheduled settlement monitor kare. Settlement webhook/report se provider transactionId, amount aur bank/provider status save karo. Daily reconciliation mismatch ko admin queue mein lao.

4. Ek customer payment, do restaurants: example

Example sirf samjhane ke liye hai; taxes, gateway fee aur actual contract rules ko is simple calculation se alag treat kiya gaya hai.

Component	Amount	Kiske liye / note
Restaurant A items	₹300	SubOrder A; Restaurant A ke vendorId se link
Restaurant B items	₹200	SubOrder B; Restaurant B ke vendorId se link
Delivery fee collected	₹40	Rider earning/margin policy se allocate hoga
Platform fee	₹10	Customer se collected, as disclosed at checkout
Platform-funded coupon	−₹50	Platform marketing expense; vendor payout ko tabhi reduce karo jab coupon vendor-funded ho
Customer total (simplified)	₹500	₹300 + ₹200 + ₹40 + ₹10 − ₹50

Allocation	Illustrative calculation	Record
Restaurant A payable	₹300 − 10% commission (₹30) = ₹270	Allocation row for SubOrder A / Vendor A
Restaurant B payable	₹200 − 10% commission (₹20) = ₹180	Allocation row for SubOrder B / Vendor B
Rider earning	Example ₹35 out of ₹40 delivery fee	Separate rider earning + payout record
Platform before gateway costs/tax	₹50 commission + ₹10 fee + ₹5 delivery margin − ₹50 coupon = ₹15	Platform ledger; not equal to gross customer payment

How correct vendor gets the money

Backend SubOrder A ki allocation ko Vendor A ke provider-side linked/vendor accountId ke saath map karega aur SubOrder B ko Vendor B ke accountId ke saath. Provider split/transfer API ko stable provider account IDs milte hain. Naam ke basis par payment route mat karo. Provider ke rules ke mutabik transfer payment capture ke baad initiate hoga ya settlement schedule par process hoga [1][2].

5. Har app ke liye alag payment track

App / party	Payment/earning flow	Admin ka control
Customer	Checkout -> provider payment -> verified capture -> order confirmation. Refund ho to payment/refund records link hon.	Failed/pending payments, duplicate attempts, refunds, disputes, receipts and reconciliation.

App / party	Payment/earning flow	Admin ka control
Restaurant / general vendor	Vendor-specific sub-orders -> commission snapshot -> net payable -> provider linked account settlement.	KYC/onboarding status, commission plan, settlement schedule/hold if supported, failed transfer and statement.
Tiffin House	One-time meal order same sub-order flow. Weekly/monthly plan ke liye subscription/recurring billing ya prepaid-cycle records, provider support ke mutabik.	Plan/payment cycle, missed/failed cycle, cancellation, vendor payable and subscription refund rules.
Rider	Per-delivery earning ledger; delivery completed/verified hone ke baad payable. Incentive/bonus/adjustment separately recorded.	Rider KYC/payout destination, approved earning rules, pending/paid/failed payouts and disputes.
FoodMitra platform	Commission + disclosed platform fee + any permitted margin; coupon cost, gateway charges and tax alag ledger accounts.	Revenue report, vendor liabilities, rider liabilities, refunds, fees, tax/reporting and cash reconciliation.

6. Database structure (recommended collections/tables)

Collection	Important fields (illustrative)	Purpose
vendors	_id, type: restaurant/tiffin_house, status, providerVendorId/linkedAccountId, kycStatus, commissionPlanId	Har payout receiver ki stable identity.
orders	_id, customerId, totalPaise, currency, paymentStatus, orderStatus, addressSnapshot	Customer checkout/parent order.
subOrders	_id, orderId, vendorId, itemSnapshot[], subtotalPaise, vendorStatus, deliveryId	Har seller/restaurant/tiffin-house ka fulfilment unit.
payments	_id, orderId, providerOrderId, providerPaymentId, amountPaise, currency, status, idempotencyKey, capturedAt	Gateway collection ka source record.
paymentAllocations	_id, paymentId, subOrderId, vendorId, grossPaise, commissionPaise, discountFunding, taxPaise, netPayablePaise	Ek payment ko vendor-wise divide karne ka immutable calculation.
ledgerEntries	_id, transactionId, accountCode, debitPaise, creditPaise, referenceType, referenceId, createdAt	Auditable money movement; corrections new reversal entries se.
settlements	_id, vendorId, allocationIds[], providerTransferId, amountPaise, status, period, failureCode, settledAt	Vendor payout/settlement tracking.
riderEarnings / riderPayouts	riderId, deliveryId, baseEarningPaise, bonusPaise, adjustmentPaise, payoutId, status	Rider earning aur bank payout ko alag track karna.
refunds	_id, paymentId, orderId/subOrderId, amountPaise, reason, providerRefundId, status	Full/partial refund and its status.
webhookEvents	provider, eventId (unique), signatureVerified, payloadReference, processingStatus, processedAt	Duplicate webhook safely ignore karne aur debugging ke liye.
auditLogs	actorId, role, action, entityType, entityId,	Admin changes ka trace.

Collection	Important fields (illustrative)	Purpose
	before/after summary, timestamp, requestId	

Store money as paise

Amounts ko integer paise mein store karo (₹125.50 = 12550), floating-point rupees mein nahi. Har payment/allocation row par currency (e.g. INR) explicitly store karo. Provider fees/taxes aur vendor-funded/platform-funded discounts ko separate fields/accounts mein rakho.

7. Statuses ko mix mat karna

Object	Suggested status path	Rule
Payment	created -> pending -> authorized -> captured; or failed/cancelled	Only verified provider event/API response decides captured state.
Sub-order	pending_payment -> paid -> accepted -> preparing -> ready -> picked_up -> delivered	Fulfilment state payment state se alag hai.
Allocation	calculated -> payable_pending -> eligible -> transferred/settled	Vendor ke liye due amount ka record, transfer status se alag.
Settlement / payout	queued -> submitted -> processing -> settled; or failed/reversed/on_hold	Failed payout ko retry karte waqt duplicate transfer guard mandatory.
Refund	requested -> processing -> succeeded/failed; partial/full	Refund success tab mark karo jab provider confirm kare.

8. Admin panel mein kya-kya hona chahiye

Admin module	Features
Payment Overview	Gross collections, captured payments, pending/failed payments, refunds, total vendor liabilities, rider liabilities, platform revenue.
Orders & Sub-orders	Parent checkout aur each restaurant/tiffin-house sub-order ka relation; payment ID, vendor, rider, statuses.
Vendor onboarding	Vendor category, provider linked account/vendor ID, KYC/onboarding state, bank verification/provider status. Sensitive bank data ko minimize karo.
Commission & pricing policy	Commission by vendor/category/order type; effective-from date; discount funding owner; delivery and platform fee rules.
Settlement queue	Payable, eligible, on hold, processing, settled, failed. Failed transfers ke error codes aur safe retry.
Refunds & disputes	Full/partial refund, reason, affected sub-order(s), vendor allocation reversal/adjustment and provider refund status.
Rider earnings	Delivery-wise earning detail, incentive, adjustments, eligible

Admin module	Features
	balance, payout batch/status.
Tiffin-house billing	One-time orders vs subscription/prepaid cycle; collection status and corresponding vendor settlement.
Reconciliation	Provider settlement report vs internal payments/ledger; mismatches, duplicate events, fee differences and unresolved items.
Audit & permissions	Role-based access; important settlement/refund/rule changes with actor, timestamp, reason and before/after values.

9. Refunds, cancellations aur failed payments

- Payment failed before capture: order ko unpaid/pending rakho; vendor payable create/settle mat karo. Expired checkout ko safely close karo.
- Customer paid but vendor cancels: refund eligibility calculate karo; provider refund initiate karo; affected allocation reverse/adjust karo. Refund final status provider confirmation se update ho.
- Partial refund: specific items/sub-order se amount map karo; vendor payable aur commission adjustment configured refund policy ke mutabik recalculate/record karo.
- Vendor ko already settlement mil chuka hai: refund ko blindly platform cash se cover karne se pehle contract/provider flow check karo. Supported transfer reversal/refund-adjustment use karo ya next settlement se valid receivable adjust karo—accounting approval aur provider rules ke mutabik [1][2].
- Webhook duplicate/retry: provider eventId unique rakho. Ek event ko doosri baar process karke customer, vendor ya rider ko double credit mat do.
- Transfer/payout failed: settlement status failed set karo, provider failure code save karo, reason resolve karke guarded retry karo. Naya duplicate payout create karne se pehle existing provider status query karo.
- Chargeback/dispute: provider caseId, deadline, evidence and amount ko track karo; disputed amount ko reporting mein clear flag karo.

10. Multiple users ek saath pay karein to system safe kaise rahega?

- Payment creation par unique idempotency key use karo, e.g. checkoutId + attemptVersion. Double-tap se 2 provider orders na banne do.
- Database mein unique indexes rakho: provider payment ID, provider webhook event ID, idempotency key, provider transfer ID (where appropriate).
- Webhook handler short, retry-safe aur idempotent ho. Pehle event persist karo, signature verify karo, phir transactionally internal state update karo.
- Same order ki do competing payment attempts mein captured payment ko deterministic tareeke se identify karo; extra captured payment aaye to automatically second fulfilment na banao—review/refund workflow trigger karo.
- Financial writes ke liye database transaction/consistent locking strategy use karo. Multi-document transactions ke liye MongoDB deployment transaction support (replica set) verify karo.
- Ledger entries immutable rakho. Galti correct karne ke liye old row edit/delete karne ki jagah reversal/adjustment entry banao.
- Secrets only server environment/secret manager mein; logs mein private keys, OTPs, full payment credentials ya sensitive bank data print na karo.
- Admin permissions: viewer, support, finance maker, finance approver, super admin. Large refunds/payout changes ke liye maker-checker approval rakho.

11. Backend APIs (illustrative names)

Method + path	Purpose / permission
POST /api/checkout	Cart validate, sub-orders and immutable pricing snapshot create.
POST /api/payments/create-order	Backend provider order/session create; idempotent.
POST /api/webhooks/payment-provider	Payment/transfer/refund provider webhook; signature verification required.
GET /api/payments/:paymentId	Customer ko apni payment ka safe status.
GET /api/vendors/me/earnings	Vendor ko apna payable/settled statement only.
GET /api/riders/me/earnings	Rider ko apni delivery-wise earnings only.
GET /api/tiffin-house/me/settlements	Tiffin house ko apne orders/subscriptions/settlements.
GET /api/admin/payments	Admin-authorized filtered payment search.
GET /api/admin/settlements	Admin settlement queue/report.
POST /api/admin/refunds	Permission + reason + idempotency + provider refund API.
POST /api/admin/settlements/:id/retry	Only failed eligible transfer, after provider status check and approval.
GET /api/admin/reconciliation	Provider reports vs internal ledger mismatch queue.

Actual route names tumhare existing Express version ke according adjust honge. Provider ke secret API calls hamesha Node backend se karo; React Native app se direct transfer/refund create na karao.

12. Implementation sequence — pehle kya, phir kya

Phase 1 — Business rules: Commission, coupon funding, rider earning, refund/cancellation, settlement eligibility, tiffin cycle aur payout rules decide karo; CA/provider se validate karo.
Phase 2 — Provider & onboarding: Marketplace provider evaluate karo. Split support, vendor KYC, rider payout, settlement/hold rules, refunds, webhooks, fees aur account approval confirm karo.
Phase 3 — Database: Vendors/provider IDs, parent orders, sub-orders, payments, allocations, ledger, settlements, refunds, webhook events aur audit collections banao.
Phase 4 — Customer collection: Checkout validation, provider order creation, payment callback aur backend status query banao. App callback alone se paid status mat set karo.
Phase 5 — Webhooks: Signature verify, unique event ID, duplicate-safe processing, captured/failed events aur retries implement karo.
Phase 6 — Allocation + ledger: Vendor net, commission, fees, coupon funding, tax aur rider earning calculate karo; integer paise aur auditable ledger rakho.
Phase 7 — Vendor/tiffin settlement: Vendor onboarding, split/transfer APIs, settlement webhooks aur failed-transfer workflow banao.
Phase 8 — Rider payout: Delivery verify karo; earning rules, payout batches/API, failure handling aur rider statements add karo.
Phase 9 — Refund/dispute: Full/partial refunds, cancellations, reversals/next-settlement adjustments aur disputes track karo.
Phase 10 — Admin dashboard: Overview, vendors, commission, payments, settlements, refunds, rider earnings, tiffin cycles, reconciliation aur audit trail.
Phase 11 — QA + launch: Sandbox tests, security review, reconciliation trial, small vendor pilot, then monitored phased rollout.

13. Launch se pehle ye tests zaroor karo

- 2 customers same waqt payment karein: payments/orders cross-link na hon.
- One checkout mein Restaurant A + B items: har allocation correct vendor accountId se linked ho.
- Customer success screen dikhaye, lekin provider payment fail ho: order paid na ho.
- Same successful webhook 2–3 baar aaye: payment/allocation/settlement double create na ho.
- Vendor cancel after payment: customer refund and vendor payable adjustment correct ho.
- Partial refund of one item in a multi-vendor checkout: correct sub-order/vendor impact ho.
- Vendor payout failed/timeout: duplicate transfer na ho; admin retry safe ho.
- Provider settlement report mein net amount/fees mismatch: reconciliation queue mein dikhe.
- Tiffin subscription cycle renew fails/cancels: cycle status and vendor amount accurate rahe.
- Rider delivery incomplete/cancelled: earning policy ke mutabik calculate ho.
- Unauthorized vendor customer, another vendor ka settlement/earnings na dekh paaye.
- Admin refund/commission/payout change audit log mein record ho.

14. Final recommended operating model

CUSTOMER PAYS -> PAYMENT PROVIDER CAPTURES + WEBHOOK CONFIRMS
BACKEND MAPS PAYMENT -> PARENT ORDER + VENDOR-WISE SUB-ORDERS
ALLOCATION ENGINE -> VENDOR NET + PLATFORM SHARE + DELIVERY/RIDER EARNING
IMMUTABLE LEDGER -> WHO OWES WHOM, HOW MUCH, AND WHY
FULFILMENT / REFUND RULES -> SETTLEMENT ELIGIBILITY
PROVIDER SPLIT / TRANSFER + RIDER PAYOUT FLOW
WEBHOOK + PROVIDER REPORT -> SETTLED / FAILED / REVERSED
ADMIN RECONCILIATION -> AUDITABLE REPORTS AND CONTROL

Non-negotiable rule

Admin dashboard ka role payment ko control/monitor karna hai, money movement ko fake karna nahi. Har real transfer/refund provider API se ho, har status provider response/webhook aur reconciliation se confirm ho, aur har amount ka internal ledger reference ho.

15. Official provider references

[1] Razorpay Route — marketplace linked accounts, transfers, settlement, refunds and reconciliation: Official Route documentation

More detail on linked account transfers and settlement: Razorpay Route transfer docs

[2] Cashfree Easy Split — vendor onboarding, split orders, settlements, refunds and reconciliation: Easy Split API overview

Product overview and marketplace split capabilities: Cashfree Easy Split

Before production

Provider products, eligibility, settlement holds, KYC, fees, supported account types and policies can vary by account and contract. Before writing the final transfer integration, contact the provider and your finance/legal advisor to confirm the allowed funds flow and compliance/tax treatment for FoodMitra.